# CSE331: Microprocessor Interfacing and Embedded Systems

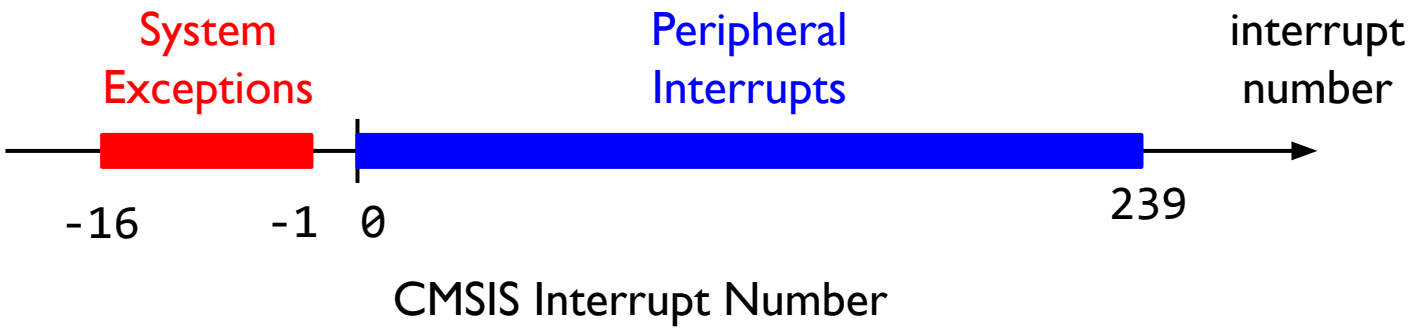
## Lecture#12: Interrupt Part#2 (Chapter#11)

Summer, 2023

Mohammad A. Qayum, Ph.D.
ECE, NSU, Dhaka, Bangladesh

# Interrupt Number

- Cortex-M supports up to 256 interrupts.



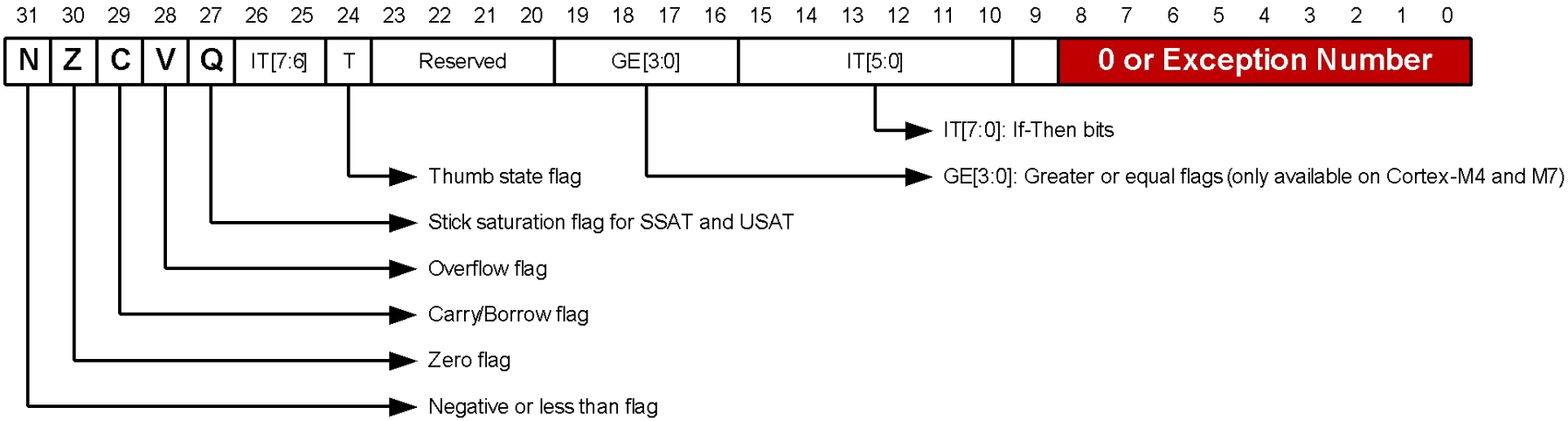

- First 16 are system exceptions
  - CMSIS defines their interrupt numbers as negative
  - Defined by ARM core
- The rest 240 are peripheral interrupts
  - Peripheral interrupt number starts with 0.
  - Defined by chip manufacturers.

# Interrupt Number in CMSIS *vs* in PSR

## Interrupt number for CMSIS functions

```
NVIC_DisableIRQ (IRQn);              // Disable interrupt
NVIC_EnableIRQ (IRQn);               // Enable interrupt
NVIC_ClearingPending (IRQn);         // clear pending status
NVIC_SetPriority (IRQn, priority); // set priority level
```

## Interrupt number in Program Status Register (PSR)




**Interrupt Number in PSR = 16 + Interrupt Number for CMSIS**

# CMSIS Interrupt Number

```
/******  Cortex-M4 System Exceptions ********************************************************/
  NonMaskableInt_IRQn    = -14,    /* 2 Cortex-M4 Non Maskable Interrupt                    */
  HardFault_IRQn         = -13,    /* 3 Cortex-M4 Hard Fault Interrupt                      */
  MemoryManagement_IRQn  = -12,    /* 4 Cortex-M4 Memory Management Interrupt               */
  BusFault_IRQn          = -11,    /* 5 Cortex-M4 Bus Fault Interrupt                       */
  UsageFault_IRQn        = -10,    /* 6 Cortex-M4 Usage Fault Interrupt                     */
  SVCall_IRQn            = -5,     /* 11 Cortex-M4 SV Call Interrupt                        */
  DebugMonitor_IRQn      = -4,     /* 12 Cortex-M4 Debug Monitor Interrupt                  */
  PendSV_IRQn            = -2,     /* 14 Cortex-M4 Pend SV Interrupt                        */
  SysTick_IRQn           = -1,     /* 15 Cortex-M4 System Tick Interrupt                    */
/******  Peripheral Interrupt Numbers ********************************************************/
  WWDG_IRQn              = 0,      /* Window WatchDog Interrupt                             */
  PVD_PVM_IRQn           = 1,      /* PVD/PVM1,2,3,4 through EXTI Line detection Interrupts */
  TAMP_STAMP_IRQn        = 2,      /* Tamper and TimeStamp interrupts through the EXTI line */
  RTC_WKUP_IRQn          = 3,      /* RTC Wakeup interrupt through the EXTI line            */
  FLASH_IRQn             = 4,      /* FLASH global Interrupt                                */
  RCC_IRQn               = 5,      /* RCC global Interrupt                                  */
  EXTI0_IRQn             = 6,      /* EXTI Line0 Interrupt                                  */
  ...
```

**System Exceptions**

**Peripheral Interrupts**

stm32l476xx.h

# Enable an Interrupt

- Enable a system exception
  - Some are always enabled (cannot be disabled)
  - No centralized registers for enabling/disabling
  - Each are control by its corresponding components, such as SysTick module

- Enable a peripheral interrupt
  - Centralized register arrays for enabling/disabling
  - **`ISER`** registers for enabling
  - **`ICER`** registers for disabling

# Enabling Peripheral Interrupts

**Interrupt Set Enable Register 0 (ISER0)**

| | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Enable Bit | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Interrupt Number | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| | I2C1_EV | TIM4 | TIM3 | TIM2 | TIM11 | TIM10 | TIM9 | LCD | EXTI9_5 | COMP | DAC | USB_LP | USB_HP | ADC1 | DMA1_CH7 | DMA1_CH6 | DMA1_CH5 | DMA1_CH4 | DMA1_CH3 | DMA1_CH2 | DMA1_CH1 | EXTI4 | EXTI3 | EXTI2 | EXTI1 | EXTI0 | RCC | FLASH | RTC_WKUP | TAMPER_STAMP | PVD | WWDG |

**Interrupt Set Enable Register 1 (ISER1)** ***Address of ISER1 = Address of ISER0 + 4***

| | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Enable Bit | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Interrupt Number | | | | | | | | | | | | | | | | | | | | 44 | 43 | 42 | 41 | 40 | 39 | 38 | 37 | 36 | 35 | 34 | 33 | 32 |
| | | | | | | | | | | | | | | | | | | | | TIM7 | TIM6 | USB_FS_WKUP | RTC_Alarm | EXTI15_10 | USART3 | USART2 | USART1 | SPI2 | SPI1 | I2C2_ER | I2C2_EV | I2C1_ER |

```
TIM7_IRQn = 44

NVIC->ISER[1] = 1 << 12;      // Enable Timer 7 interrupt
```

# Disabling Peripheral Interrupts

**Interrupt Clear Enable Register 0 (ICER0)**

| | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Clear Enable Bit | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Interrupt Number | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| | I2C1_EV | TIM4 | TIM3 | TIM2 | TIM11 | TIM10 | TIM9 | LCD | EXTI9_5 | COMP | DAC | USB_LP | USB_HP | ADC1 | DMA1_CH7 | DMA1_CH6 | DMA1_CH5 | DMA1_CH4 | DMA1_CH3 | DMA1_CH2 | DMA1_CH1 | EXTI4 | EXTI3 | EXTI2 | EXTI1 | EXTI0 | RCC | FLASH | RTC_WKUP | TAMPER_STAMP | PVD | WWDG |

**Interrupt Clear Enable Register 1 (ICER1)** ***Address of ICER1 = Address of ISER0 + 4***

| | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Clear Enable Bit | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Interrupt Number | | | | | | | | | | | | | | | | | | | | 44 | 43 | 42 | 41 | 40 | 39 | 38 | 37 | 36 | 35 | 34 | 33 | 32 |
| | | | | | | | | | | | | | | | | | | | | TIM7 | TIM6 | USB_FS_WKUP | RTC_Alarm | EXTI15_10 | USART3 | USART2 | USART1 | SPI2 | SPI1 | I2C2_ER | I2C2_EV | I2C1_ER |

```
TIM7_IRQn = 44

NVIC->ICER[1] = 1 << 12;      // Diable Timer 7 interrupt
```

# Disable/Enable Peripheral Interrupts

- For all peripheral interrupts: `IRQn ≥ 0`
- Method 1:
  - `NVIC_EnableIRQ (IRQn);  // Enable interrupt`
  - `NVIC_DisableIRQ (IRQn); // Disable interrupt`
- Method 2:
  - Enable:
    - `NVIC->ISER[ IRQn / 32] = 1 << (IRQn % 32);`
    - Better solution:
    - `NVIC->ISER[ IRQn >> 5] = 1 << (IRQn & 0x1F);`
  - Disable:
    - `NVIC->ICER[ IRQn >> 5] = 1 << (IRQn & 0x1F);`

# Interrupt Priority

- Inverse Relationship:
  - Lower priority value means higher urgency.
    - Priority of Interrupt A = 5,
    - Priority of Interrupt B = 2,
    - B has a higher priority/urgency than A.

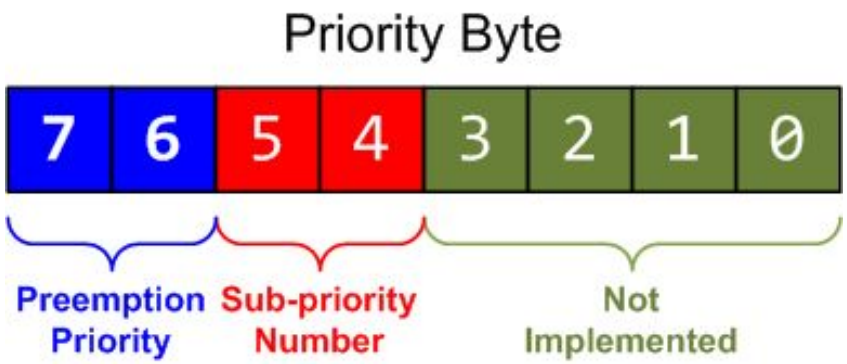
- Fixed priority for Reset, HardFault, and NMI.

| Exception | IRQn | Priority |
|---|---|---|
| Reset | N/A | -3 (the highest) |
| Non-maskable Interrupt (NMI) | -14 | -2 (2nd highest) |
| Hard Fault | -13 | -1 |

- Adjustable for all the other interrupts

# Interrupt Priority

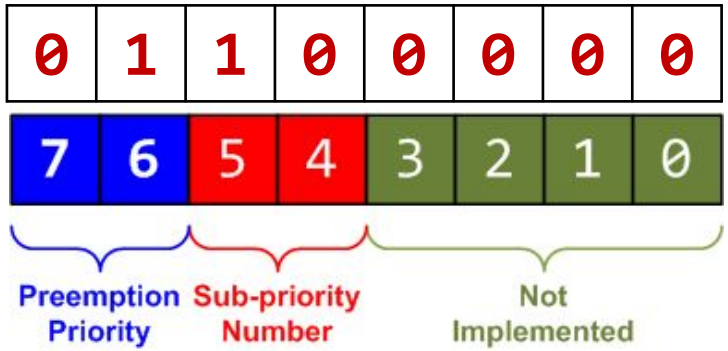
- Interrupt priority is configured by Interrupt Priority Register (IP)
- Each priority consists of two fields, including preempt priority number and sub-priority number.
  - The preempt priority number defines the priority for preemption.
  - The sub-priority number determines the order when multiple interrupts are pending with the same preempt priority number.




default setting

# Interrupt Priority Levels

```
NVIC_SetPriority(7, 6);
```




```
IP = 0x60 = 96
```

core_cm4.h or core_cm3.h

```
typedef struct {
  ...
  // Interrupt Priority Register
  volatile uint8_t IP[240];
  ...
} NVIC_Type;
```

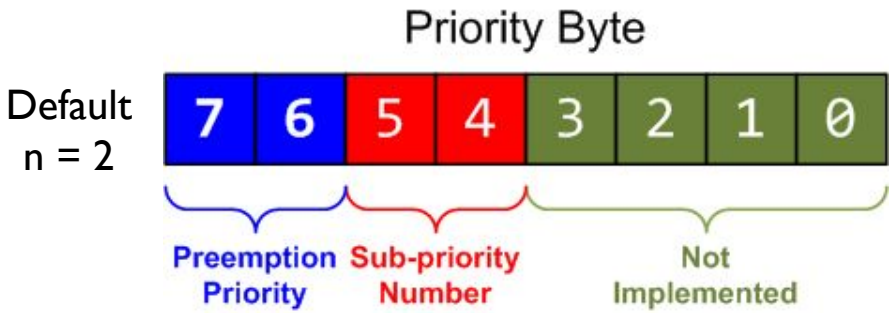
It is equivalent to:

```
NVIC->IP[7] = (6 << 4) & 0xff;
```

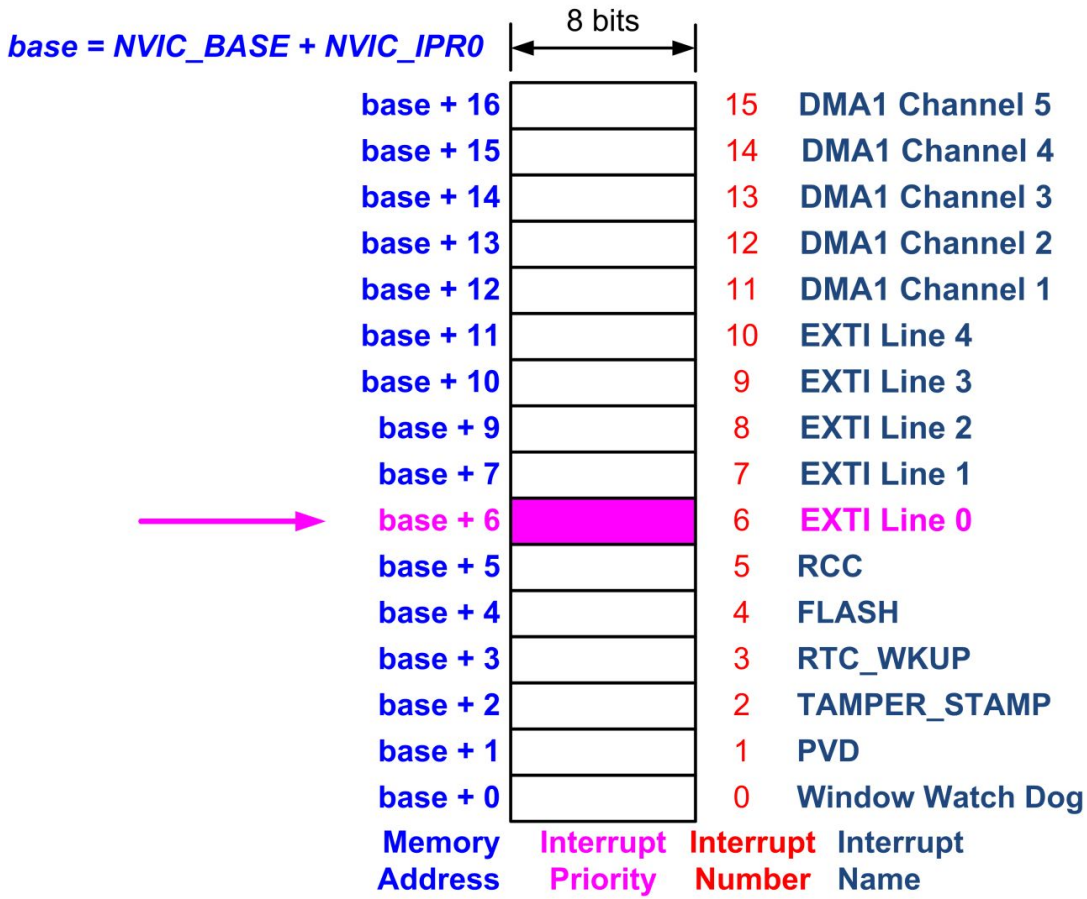
# Preemption and Sub-priority Configuration

- `NVIC_SetPriorityGrouping(n)`
  - Perform unlock, and update AIRCR register

| n | # of bits in preemption priority | # of bits in sub-priority |
|---|---|---|
| 0 | 0 | 4 |
| 1 | 1 | 3 |
| 2 (default) | 2 | 2 |
| 3 | 3 | 1 |
| 4 | 4 | 0 |

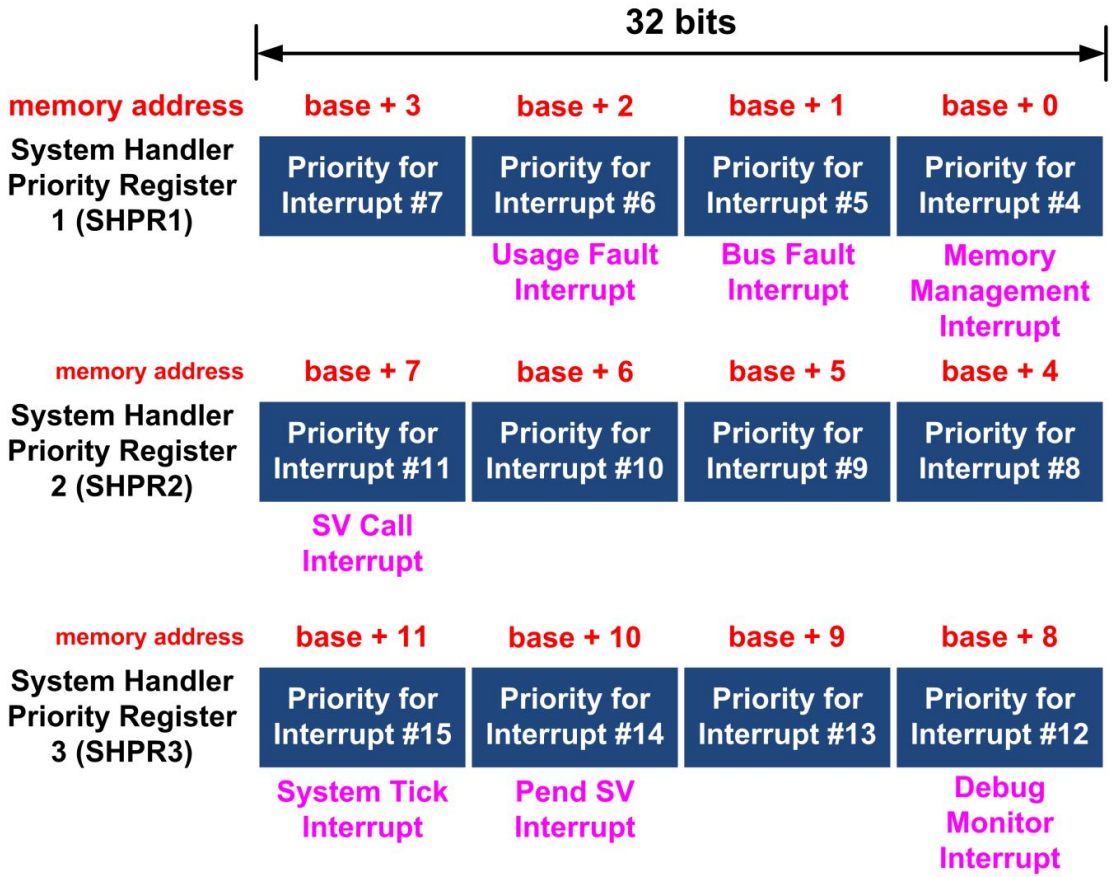


# Priority of Peripheral Interrupts




```
// Set the priority for EXTI 0 (Interrupt number 6)
NVIC->IP[6] = 0xF0;
```

# Priority of System Interrupts




```
// Set the priority of a system interrupt IRQn
SCB->SHP[(IRQn) & 0xF) - 4] = (priority << 4) & 0xFF;
```

# Masking Priority

- Disable all interrupts with less urgency
  - For critical code, only certain interrupts are allowed.
  - BASEPRI: Disable all interrupts of specific priority level or higher priority level

```
// Disable interrupts with priority value same or higher
__set_BASEPRI( 5 << 4 )

// Critical code
...

// Remove BASEPRI masking
__set_BASEPRI(0);
```

# Exception-masking registers (PRIMASK, FAULTMASK and BASEPRI)

- PRIMASK: Used to disable all exceptions except Non-maskable interrupt (NMI) and hard fault.
  - Write 1 to PRIMASK to disable all interrupts except NMI

```
MOV R0, #1
MSR PRIMASK, R0
```

  - Write 0 to PRIMASK to enable all interrupts

```
MOV R0, #0
MSR PRIMASK, R0
```

- FAULTMASK: Like PRIMASK but change the current priority level to -1, so that even hard fault handler is blocked

- BASEPRI: Disable interrupts only with priority lower than a certain level
  - Example, disable all exceptions with priority level higher than 0x60

```
MOV R0, #0x60
MSR BASEPRI, R0
```